



# Automated Red Teaming for Generative Models: A Survey

Anonymous Author(s)<sup>1,\*</sup>

1. Anonymous Institution

Received month dd, yyyy; accepted month dd, yyyy

E-mail: anonymous@example.com. \* These authors contributed equally to this work.

© Higher Education Press 2026

## Abstract

The rapid deployment of generative models has created a need for automated red-teaming methods that discover failures beyond manually authored test cases. This survey focuses on automated red-teaming systems that generate, execute, and evaluate adversarial or safety-relevant interactions with generative models and model-based applications. We study automated red teaming across large language models, vision-language models, text-to-image and text-to-video systems, retrieval-augmented applications, and tool-using agents. We organize the literature around a closed-loop pipeline that connects risk objectives, test-case generation, interaction and execution, outcome judgment, failure triage, and mitigation. Within this pipeline, we unify red-teaming strategies by their search space, optimization signal, access assumptions, interaction horizon, and modality. We then compare evaluation metrics, benchmarks, automated judges, human-in-the-loop protocols, and reporting practices, with particular attention to attack success, harm severity, coverage, transferability, cost, and over-refusal. Finally, we identify open problems in long-horizon interaction, multimodal and multilingual testing, reliable evaluation, adaptive coverage, reproducibility, and the responsible use of automated red-teaming systems. The survey is intended to provide a common vocabulary and a practical roadmap for building scalable, evidence-based automated red-teaming evaluations.

## Key words

automated red teaming; generative models; large language models; vision-language models; text-to-image generation; AI safety; adversarial evaluation; jailbreaks

## ■ 1 Introduction

### 1.1 Motivation: Problem Scope

- Why red teaming matters.

Generative models can fail under inputs that are rare, indirect, context-dependent, or outside the distribution represented by standard safety datasets. Red teaming treats safety assessment as proactive discovery: a tester constructs situations that may expose harmful compliance, privacy leakage, policy evasion, unsafe tool use, or other undesirable behavior before deployment.

- Why automation matters.

Manual red teaming provides valuable expertise but is expensive, difficult to reproduce, and limited in coverage. Automated red teaming extends this process by generating test cases, searching through interaction states, executing attacks, judging outputs, and retaining failures as regression tests. The central challenge is not merely to maximize attack success, but to discover diverse, relevant, and verifiable failure modes.

### 1.2 Survey Positioning: Research Questions

- Scope.

This survey focuses on automated or partially automated red-teaming systems for generative models. LLMs, VLMs/MLLMs, T2I/T2V systems, RAG applications, and tool-using agents are treated as target

systems. General alignment, generic adversarial examples, and static safety benchmarks are included only when they support an automated red-teaming component.

- Research questions.

The survey is organized around five questions: how red-team objectives are specified; how test cases are generated and searched; how interactions and failures are evaluated; how methods transfer across models, modalities, and applications; and how discovered failures improve subsequent mitigation and regression testing.

- Relation to prior surveys.

Prior surveys provide complementary views of generative-model red teaming, LLM security, attack evaluation, practical red-teaming systems, and multimodal safety [1–5]. We use these works as field-level anchors, then reorganize the evidence around automation boundaries, closed-loop feedback, and target-system transfer.

### 1.3 Contributions: Article Roadmap

- Contributions.

The survey makes four contributions:

- a closed-loop framework that connects risk specification, attack generation, execution, judgment, triage, and mitigation;
- a taxonomy of automated generation methods based on search space, feedback, access, interaction, and modality;

- a comparison of evaluators, metrics, benchmarks, and reporting protocols; and
- a research agenda for coverage, reliability, multimodality, long-horizon testing, and responsible use.

- Article roadmap.

Section 2 describes the survey protocol. Section 3 defines the threat model and closed-loop framework. Section 4 reviews automated generation and search. Section 5 discusses judgment, metrics, and benchmarks. Section 6 organizes evidence by target system. Section 7 connects failures to mitigation. Section 8 covers tools, human expertise, and responsible use. Section 9 presents open problems.

## ■ 2 Survey Methodology: Scope

### 2.1 Literature Protocol: Evidence Base

- Retrieval strategy.

Report the databases, search engines, venues, time range, and query terms used to collect papers. Search terms should include automated red teaming, adversarial search, jailbreak generation, safety evaluation, prompt injection, MLLM safety, T2I safety, agent security, and red-team benchmark.

- Screening procedure.

Describe deduplication, title–abstract screening, full-text screening, citation chaining, and the replacement of preprints by published versions when available.

### 2.2 Inclusion Criteria: Coding Schema

- Inclusion rule.

Include a paper when automation contributes to risk specification, test generation, adversarial search, interaction, outcome judgment, benchmark construction, failure triage, or mitigation using red-team traces. Separate direct red-teaming papers from supporting safety, jailbreak, and agent-security papers.

- Paper-level coding.

Annotate target system, risk, attacker access, generation mechanism, interaction horizon, feedback signal, evaluator, benchmark, metric, compute cost, and outcome. Use this schema to build comparison tables rather than summarizing papers as an unrelated sequence.

### 2.3 Survey Limitations: Coverage Bias

- Evidence limitations.

State the cutoff date, language restrictions, treatment of unpublished work, benchmark bias, closed-model access limitations, and the risk that rapidly changing attack methods become obsolete.

## ■ 3 Unified Automated Red-Teaming Framework

### 3.1 Definitions: System Boundaries

- Automated red teaming.

Define automated red teaming as goal-directed discovery of undesirable behavior in which one or more stages of test generation, selection, interaction, judgment, or iteration are automated under an explicit threat model.

- Target system boundary.

Separate the target model, safety layer, orchestration layer, retrieval layer, memory, tools, and deployed application. A discovered failure should be attributed to the smallest component or interaction responsible for it.

### 3.2 Threat Model: Attack Conditions

- Attacker access.

Distinguish black-box, gray-box, white-box, and application-level access. Record query limits, model knowledge, system-prompt visibility, gradient access, activation access, control over retrieved content, and control over tools or memory.

- Risk objective.

Classify objectives into harmful compliance, privacy leakage, prompt injection, policy evasion, misinformation, bias, copyright exposure, unsafe tool use, reliability failure, and over-refusal.

- Interaction horizon.

Separate single-turn, iterative, multi-turn, stateful, multi-agent, and long-horizon settings. The horizon determines whether the attack optimizes a response, a conversation trajectory, or an external action.

### 3.3 Closed-Loop Pipeline: Evaluation Cycle

- Risk specification.

Translate a policy, threat model, harm taxonomy, or application requirement into a test objective and an observable failure condition.

- Test generation.

Construct an initial population of prompts, scenarios, images, videos, retrieved documents, tool outputs, or multi-turn plans.

- Search: Interaction.

Select, mutate, optimize, or compose test cases while interacting with the target system. The search state may include prompts, conversation history, tool state, model feedback, and prior failures.

- Outcome judgment.

Determine whether the target behavior satisfies the failure condition using rules, classifiers, reward models, LLM judges, VLM judges, environments, or human annotators.

- Triage: Regression.

Cluster failures, estimate severity and novelty, remove duplicates, record reproducible attack traces, and convert confirmed failures into held-out regression tests.

## ■ 4 Automated Generation: Search Strategies

### 4.1 Scenario Construction: Initial Test Cases

- Seed prompts.

Review expert prompts, policy-derived scenarios, safety datasets, attack libraries, synthetic harmful-intent descriptions, and application-specific user stories.

- Scenario expansion.

Cover semantic paraphrasing, translation, role-play, encoding, formatting, contextual embedding, instruction indirection, and risk-conditioned generation.

- Diversity control.

Compare novelty penalties, coverage constraints, population methods, quality-diversity search, clustering, and explicit demographic, cultural, or geographic balancing.

#### 4.2 Model-Based Generation: Attacker Models

- Attacker language models.

Review zero-shot generation, proposer–judge loops, attacker–target interaction, model-written evaluations, and attacker fine-tuning.

- Reverse generation.

Discuss methods that begin from a harmful output, policy violation, or target behavior and search backward for an input that elicits it.

- Multi-agent generation.

Separate attacker, critic, judge, planner, and target roles. Analyze whether role separation improves exploration or introduces correlated errors.

#### 4.3 Algorithmic Search: Optimization Signals

- Fuzzing: Mutation.

Cover seed selection, mutation operators, random search, evolutionary search, and black-box feedback.

- Reinforcement learning.

Review attacker training with reward signals from toxicity classifiers, safety judges, target compliance, novelty, or environment outcomes.

- Gradient-Level: Token-Level: Activation-Level Optimization.

Describe white-box optimization of suffixes, tokens, continuous embeddings, image perturbations, or internal representations. Compare access requirements, transferability, realism, and cost.

#### 4.4 Adaptive Interaction: Stateful Search

- Iterative refinement.

Analyze attacks that learn from refusals, partial compliance, judge feedback, or changes in target behavior.

- Multi-turn search.

Cover trust building, context shaping, refusal recovery, conversational state, memory persistence, and long-horizon planning.

- Application-level search.

Extend the search state to retrieved documents, web pages, tool outputs, user permissions, external state, and action consequences.

### ■ 5 Evaluation: Judgment: Benchmarking

#### 5.1 Failure Definition: Success Conditions

- Behavioral success.

Distinguish policy violation, harmful content, unsafe action, privacy leakage, misinformation, reliability failure, and evaluator disagreement. Every benchmark should define the target behavior before reporting attack success.

- Severity: Usefulness.

Measure not only whether a guardrail was bypassed, but also whether the response contains actionable, relevant, and sufficiently complete harmful information or performs an unsafe action.

#### 5.2 Automated Judges: Human Agreement

- Judge families.

Compare keyword rules, lexical matching, safety classifiers, reward models, LLM-as-judge, VLM-as-judge, structured rubrics, environment success signals, and human review.

- Judge reliability.

Report calibration, inter-rater agreement, judge disagreement, false positives, false negatives, judge hacking, and agreement with human judgments.

#### 5.3 Metrics: Experimental Comparability

- Attack effectiveness.

Cover attack success rate, harmfulness score, refusal rate, compliance quality, and task-level success.

- Search quality.

Cover coverage, diversity, novelty, marginal discovery rate, transferability, robustness, query efficiency, latency, and compute cost.

- Safety–utility balance.

Report over-refusal, benign-task utility, calibration, safe completion quality, and the trade-off between harmfulness reduction and helpfulness.

#### 5.4 Benchmarks: Protocol Design

- Static datasets.

Compare behavior sets, harmful instructions, refusal suites, adversarial examples, multimodal pairs, and application-specific test collections.

- Dynamic environments.

Discuss extensible harnesses, agent environments, tool-use simulators, external-state tasks, adaptive attacks, and continual benchmark updates.

- Reproducibility checklist.

Require model version, system prompt, access condition, sampling parameters, query budget, random seeds, judge version, annotation protocol, compute cost, attack artifacts, and release policy.

### ■ 6 Automated Red Teaming Across Generative Models

#### 6.1 Large Language Models: Text Interaction

- Prompt-level attacks.

Review semantic jailbreaks, mutation-based fuzzing, role-play, encoding, multilingual prompts, prompt injection, and refusal manipulation.

- Token-level attacks.

Review adversarial suffixes, gradient-guided optimization, transfer attacks, activation steering, and access-dependent evaluation.

- Conversation-level attacks.

Analyze multi-turn persuasion, context contamination, many-shot behavior, memory persistence, and stateful safety drift.

#### 6.2 VLMs: MLLMs: Cross-Modal Inputs

- Visual attack surfaces.

Organize attacks by text-only inputs, malicious images, visual instructions, OCR, image–text mismatch, adversarial perturbations, and jointly optimized inputs.

- Cross-modal evaluation.

Track whether the failure originates in the vision encoder, fusion module, language model, safety layer, or modality interaction.

### 6.3 T2I: T2V: Diffusion Systems

- Text-to-image testing.

Cover unsafe prompt generation, safety-filter bypass, semantic obfuscation, image-space perturbation, prompt-image mismatch, and benign prompts that produce harmful images.

- Text-to-video testing.

Extend the taxonomy to temporal consistency, unsafe actions, identity and privacy risks, audio or video conditioning, and harms that emerge across frames.

- Image-aware judgment.

Require image or video evaluators and targeted human validation rather than text-only safety labels.

### 6.4 Agents: RAG: Tool-Use

- Indirect prompt injection.

Cover malicious instructions embedded in retrieved documents, web pages, files, tool outputs, and external data.

- Tool misuse.

Evaluate privilege escalation, unsafe planning, unauthorized actions, data exfiltration, and failures caused by tool output interpretation.

- Environment-level success.

Measure action consequences, security-property violations, task completion, reversibility, and user impact rather than textual harmfulness alone.

### 6.5 Multilingual Testing: Cultural Context

- Language coverage.

Discuss low-resource languages, code-switching, translation, dialect, and language-specific safety gaps.

- Cultural coverage.

Discuss regional norms, demographic context, participatory testing, and the risk that automated generation reproduces cultural bias.

## ■ 7 Failure Remediation: Safety Improvement

### 7.1 Training-Time Mitigation: Red-Team Traces

- Safety tuning.

Review supervised fine-tuning, preference optimization, adversarial training, data filtering, and targeted training with red-team traces.

- Generalization test.

Evaluate held-out risks, unseen attack strategies, cross-model transfer, and the possibility that mitigation memorizes known attacks.

### 7.2 Inference-Time Mitigation: Guardrails

- Input protection.

Cover prompt filters, injection detection, perturbation defenses, policy classifiers, and context sanitization.

- Output protection.

Cover refusal policies, output filters, self-critique, model ensembles, decoding controls, and multimodal safety layers.

### 7.3 Application Mitigation: System Controls

- System security.

Discuss access control, least-privilege tools, retrieval sanitization, memory isolation, monitoring, human approval, and safe failure behavior.

- Model-system distinction.

Emphasize that model alignment cannot address every risk created by orchestration, data flow, permissions, or external actions.

### 7.4 Regression Testing: Continuous Red Teaming

- Failure retention.

Describe how confirmed failures become versioned regression cases with severity, provenance, reproduction steps, and ownership.

- Safety drift.

Measure changes across model updates, policy updates, guardrail updates, tool changes, retrieval changes, and judge changes.

## ■ 8 Practice: Tools: Responsible Use

### 8.1 Frameworks: Evaluation Harnesses

- Framework capabilities.

Compare model adapters, attack plugins, judges, logging, parallel execution, multimodal inputs, tool environments, and regression testing.

- Benchmark versus harness.

Distinguish static benchmark convenience from extensible harnesses that support adaptive attackers, application context, and reproducible experiments.

### 8.2 Human Expertise: Participatory Testing

- Human contribution.

Identify tasks that remain difficult to automate, including novel-harm discovery, contextual severity judgment, community representation, and test prioritization.

- Human-machine division.

Frame automation as a force multiplier for human expertise. Report where humans design policies, validate judges, inspect failures, or approve disclosure.

### 8.3 Artifact Safety: Disclosure Practice

- Risk controls.

Discuss access control, redaction, staged release, harmful-content handling, artifact storage, and the risk that attack prompts become reusable attack tools.

- Reporting standard.

Connect threat model, generator, execution environment, evaluator, metric, cost, failure, mitigation, and held-out regression result in one report.

## ■ 9 Open Problems: Research Agenda

### 9.1 Coverage: Discovery Guarantees

- Risk-space coverage.

Develop principled definitions of coverage, novelty, diminishing returns, and confidence that important failure modes remain undiscovered.

## 9.2 Judgment: Evaluation Reliability

- Adversary-resistant judges.

Build calibrated multimodal judges, quantify uncertainty, detect judge gaming, and combine automated judgment with targeted human review.

## 9.3 Long-Horizon Testing: Stateful Systems

- Persistent interaction.

Move beyond single-turn prompts toward memory, tools, planning, external state, and realistic environments.

## 9.4 Transfer: Cross-Modal Robustness

- Generalization across systems.

Measure whether failures transfer across model families, versions, modalities, languages, applications, and safety policies.

## 9.5 Continual Red Teaming: Evaluation Drift

- Adaptive maintenance.

Design systems that update risk taxonomies, search policies, and regression suites without silently changing the evaluation target.

## 9.6 Governance: Responsible Release

- Reproducibility: Disclosure.

Establish shared protocols for versioning, access, benchmark maintenance, artifact release, high-severity reporting, and public accountability.

## ■ 10 Conclusion

Automated red teaming should be evaluated as a closed-loop safety-evaluation system rather than as a collection of isolated jailbreak techniques. A useful survey must therefore connect risk specification, generation, search, interaction, judgment, coverage, mitigation, and responsible operational practice across generative-model targets.

## ■ Acknowledgements

The authors thank the researchers and practitioners who develop public safety benchmarks, red-teaming tools, and responsible disclosure practices.

## ■ Competing Interest

The authors declare that they have no competing interests or financial conflicts to disclose.

## ■ A Detailed Survey Protocol

### A.1 Search Strings: Sources

Add the final database list, search strings, date range, and screening flow here.

### A.2 Paper Annotation: Evidence Table

Add the complete coding fields and examples used to classify each paper.

## ■ B Extended Taxonomy Tables

Add tables comparing papers by target system, threat model, generation strategy, interaction horizon, evaluator, metrics, and release status.

## ■ C Evaluation Checklist

Add a checklist that reviewers can use to assess whether an automated red-teaming result is reproducible, valid, sufficiently broad, and responsibly reported.

## ■ References

- [1] Lin L, Mu H, Zhai Z, Wang M, Wang Y, Wang R, Gao J, Zhang Y, Che W, Baldwin T, Han X, Li H. Against the achilles' heel: A survey on red teaming for generative models. *Journal of Artificial Intelligence Research*, 2025, 82: 687–775
- [2] Das B C, Amini M H, Wu Y. Security and privacy challenges of large language models: A survey. *ACM Computing Surveys*, 2025, 57(6): 1–39
- [3] Jabbar M S, Al-Azani S, Alotaibi A, Ahmed M. Red teaming large language models: A comprehensive review and critical analysis. *Information Processing and Management*, 2025, 62(6): 104239
- [4] Purpura A, Wadhwa S, Zymet J, Gupta A, Luo A, Rad M K, Shinde S, Sorower M S. Building safe genai applications: An end-to-end overview of red teaming for large language models. In: *Proceedings of the 5th Workshop on Trustworthy NLP*. 2025, 335–350
- [5] Liu X, Zhu Y, Lan Y, Yang C, Qiao Y. Safety of multimodal large language models on images and texts, 2024